POLITÉCNICA
"Ingeniamos el futuro"

CAMPUS
DE EXCELENCIA
INTERNACIONAL

Front-end para Móviles (FEM)

Tema 3 – Persistencia de Datos



miw.etsisi.upm.es

F. Javier Gil

Tema 3: Persistencia de Datos

- 3.1: Almacenamiento interno: ficheros
  - Manejo de ficheros en Android
- 3.2: Preferencias
  - Shared Preferences
- 3.3: Almacenamiento externo: tarjeta SD
- 3.4: Bases de Datos
  - SGBD SQLite
  - Room
- 3.5: Almacenamiento compartido: Proveedores de Contenidos

Almacenamiento interno: Ficheros

- apertura ficheros (propios de la aplicación))
  - lectura: `FileInputStream fis = openFileInput(NOMBRE_FICH);`
  - escritura: `FileOutputStream fos = openFileOutput(NOMBRE_FICH, modo);`
    - modo
      - `Context.MODE_PRIVATE` (elimina contenido anterior)
      - `Context.MODE_APPEND` (añade al final)
- cierre fichero: `miFichero.close();`
- localización física: `getFilesDir().getAbsolutePath()` (`/data/data/<paquete>/files`)
- recursos fichero (sólo lectura): `/res/raw`
  - contenidos estáticos para las aplicaciones
    - `InputStream fis = getResources().openRawResource(R.raw.fichero);`

https://developer.android.com/guide/topics/data/data-storage.html#filesInternal

Almacenamiento interno: Ficheros

- ejemplo escritura

```
// abrir un fichero para añadir una cadena
String cadena = "Contenido cadena\n";
FileOutputStream fos =
      openFileOutput(MI_FICHERO, Context.MODE_APPEND)
fos.write(cadena.getBytes());
fos.close();
```

- ejemplo lectura

```
// lectura contenido
BufferedReader fin = new BufferedReader(
   new InputStreamReader(openFileInput(MI_FICHERO)));
String linea = fin.readLine();
while (linea != null) {
        // añadir el contenido de linea al destino
         linea = fin.readLine();
}
fin.close();
```

Tema 3: Persistencia de Datos

- 3.1: Almacenamiento interno: ficheros
  - Manejo de ficheros en Android
- 3.2: Preferencias
  - Shared Preferences
- 3.3: Almacenamiento externo: tarjeta SD
- 3.4: Bases de Datos
  - SGBD SQLite
  - Room
- 3.5: Almacenamiento compartido: Proveedores de Contenidos

Preferencias

- mecanismo ligero de almacenamiento
- añadir biblioteca `androidx.preference`
- parejas `clave-valor` de tipos primitivos
  - definición: `res/xml/settings.xml`
- los datos son persistentes
  - `/data/data/<paquete>/shared_prefs`
  - privados de una actividad, o bien
  - compartidos entre las actividades de la aplicación

```
SharedPreferences sharedPrefs =
    PreferenceManager.getDefaultSharedPreferences(context);
prefStrValue  = sharedPrefs.getString(key, defaultValue);
prefBoolValue = sharedPrefs.getBoolean(key, defaultValue);
```

https://developer.android.com/guide/topics/ui/settings

Tema 3: Persistencia de Datos

- 3.1: Almacenamiento interno: ficheros
  - Manejo de ficheros en Android
- 3.2: Preferencias
  - Shared Preferences
- 3.3: Almacenamiento externo: tarjeta SD
- 3.4: Bases de Datos
  - SGBD SQLite
  - Room
- 3.5: Almacenamiento compartido: Proveedores de Contenidos

Persistencia de Datos: tarjeta SD

- almacenamiento externo: tarjeta SD
  - ~~se necesitan permisos en `AndroidManifest.xml`~~:

```
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

  - antes de acceder: comprobar si el almacenamiento externo es accesible: recuperar estado con `Environment.getExternalStorageState()`
    - lectura y escritura: `Environment.MEDIA_MOUNTED`
    - sólo lectura: `Environment.MEDIA_MOUNTED_READ_ONLY`
    - ficheros propios aplicación: `getExternalFilesDir()`
    - raíz tarjeta externa: `Environment.getExternalStorageDirectory()`

http://developer.android.com/guide/topics/data/data-storage.html#filesExternal

Tema 3: Persistencia de Datos

- 3.1: Almacenamiento interno: ficheros
  - Manejo de ficheros en Android
- 3.2: Preferencias
  - Shared Preferences
- 3.3: Almacenamiento externo: tarjeta SD
- 3.4: Bases de Datos
  - SGBD SQLite
  - Room
- 3.5: Almacenamiento compartido: Proveedores de Contenidos

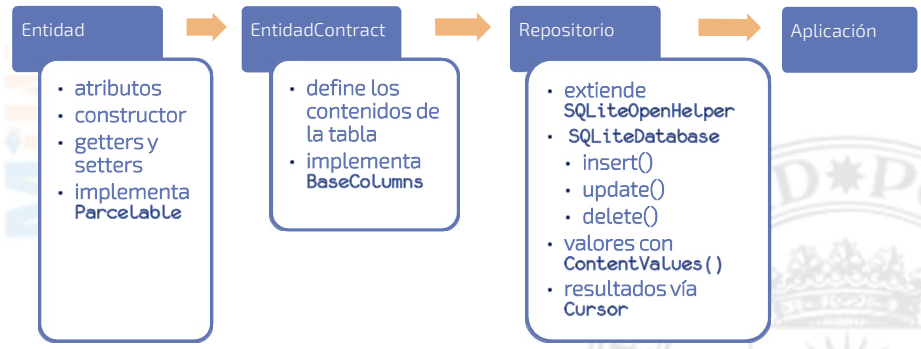
Persistencia de Datos: Bases de Datos SQLite

- Gestor de Bases de Datos Relacional: `SQLite`
  - biblioteca autocontenida, transaccional, sin necesidad de servidor, sin configuración y código libre
  - el conjunto de tablas se almacena en un fichero
  - tipado débil (los tipos se asignan a los valores, no a las columnas)
  - se almacenan en: `/data/data/<paquete>/databases/`
    - datos privados, sólo accesibles por aplicación propietaria

`https://www.sqlite.org/`

`https://developer.android.com/training/data-storage/sqlite`

Persistencia de Datos: Bases de Datos SQLite

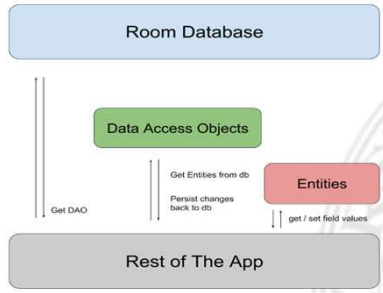


Persistencia de Datos: Bases de Datos SQLite

- <Entidad>Contract implementa BaseColumns
  - define los contenidos de la tabla
- tablas deben emplear `_id` como clave primaria
- generación de una subclase de `SQLiteOpenHelper`
  - constructor con el nombre de la DB y número de versión
  - `onCreate()` se ejecuta cuando se accede por primera vez
  - `onUpgrade()` se ejecuta cuando cambia la versión
  - clase `SQLiteDatabase`
    - proporciona `insert()`, `update()`, `delete()`, `execSQL()`, ...

Biblioteca de Persistencia de Datos Room

- proporciona una capa de abstracción sobre SQLite
  - aumenta robustez
  - mantiene la potencia de SQLite
  - aporta una caché con los datos y migraciones



https://developer.android.com/training/data-storage/room

Biblioteca de Persistencia de Datos Room

- instalación de dependencias
  - `build.gradle` de la aplicación

```
dependencies {
…

  def room_version = "2.8.2"

  implementation "androidx.room:room-runtime:$room_version"
  annotationProcessor "androidx.room:room-compiler:$room_version"

  // optional - RxJava2 support for Room
  implementation "androidx.room:room-rxjava2:$room_version"

  // optional - Guava support for Room, including Optional and ListenableFuture
  implementation "androidx.room:room-guava:$room_version"

  // optional - Test helpers
  testImplementation "androidx.room:room-testing:$room_version"
}
```

Biblioteca de Persistencia de Datos Room

- especificación de las entidades
  - cada entidad se almacena en una tabla (`@Entity`)
    - nombre de la tabla coincide con nombre de la clase
  - debe contener al menos una clave primaria (`@PrimaryKey`)
  - nombre columna: coincide con atributos (`@ColumnInfo`)

```
@Entity(tableName = "users")
public class User {
    @PrimaryKey(autoGenerate = true)
    protected Integer uid;

    @ColumnInfo(name = "first_name")
    protected String firstName;
…
```

https://developer.android.com/training/data-storage/room/defining-data

Biblioteca de Persistencia de Datos Room

- Definición de los DAOs (Data Access Object)
  - permite separar diferentes componentes de la arquitectura

```
@Dao
public interface UserDao {
    @Query("SELECT * FROM users")
    List<User> getAll();

    @Query("SELECT * FROM users WHERE uid IN (:userIds)")
    List<User> loadAllByIds(int[] userIds);

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    long[] insertAll(User... users);

    @Delete
    int delete(User user);
}
```

https://developer.android.com/training/data-storage/room/accessing-data

Biblioteca de Persistencia de Datos Room

- instanciar la base de datos

```
@Database(entities = {User.class}, version = 1)
public abstract class AppDatabase extends RoomDatabase {
    public abstract UserDao userDao();
}
```

```
AppDatabase db = Room.databaseBuilder(
        getApplicationContext(),
        AppDatabase.class,
        "users-db"
        )
        // .allowMainThreadQueries()
        // .fallbackToDestructiveMigration()
        .build();
```

Tema 3: Persistencia de Datos

- 3.1: Almacenamiento interno: ficheros
  - Manejo de ficheros en Android
- 3.2: Preferencias
  - Shared Preferences
- 3.3: Almacenamiento externo: tarjeta SD
- 3.4: Bases de Datos
  - SGBD SQLite
  - Room
- 3.5: Almacenamiento compartido: Proveedores de Contenidos

Proveedores de Contenidos

- interfaz estándar de acceso a **datos compartidos** entre aplicaciones
  - permiten gestionar el acceso a conjuntos de datos estructurados
  - proporciona mecanismos para definir la seguridad del acceso a los datos



https://developer.android.com/guide/topics/providers/content-providers

Proveedores de Contenidos: Proveedor

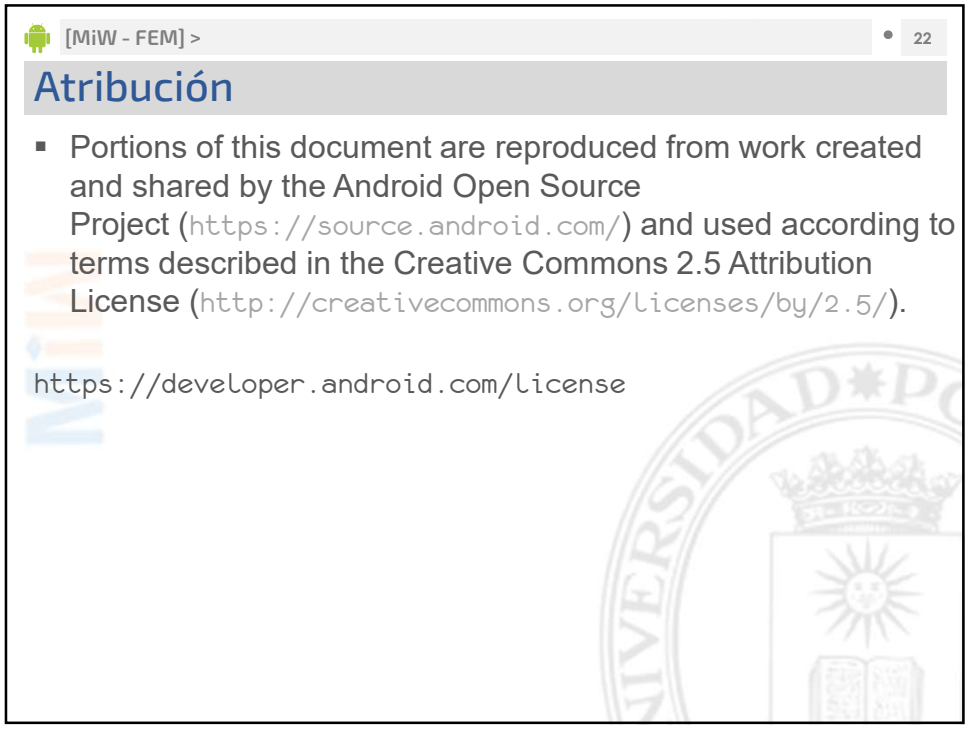
- extienden la clase `ContentProvider`
- define permisos lectura/escritura en el manifiesto
- origen de los datos
  - almacenados en ficheros (fotos, audio, video, etc.)
  - datos estructurados en una base de datos
- se debe especificar el URI de acceso al contenido:
  - `content://<authority>/<path>`
    - `content://es.upm.miw.miaplic.provider/entidad`
  - `content://<authority>/<path>/<id>`
    - `(cadena) content://es.upm.miw.miaplic.provider/entidad/*`
    - `(número) content://es.upm.miw.miaplic.provider/entidad/#`
- clase `UriMatcher`: mapea patrones URI a enteros

https://developer.android.com/guide/topics/providers/content-providers

Proveedores de Contenidos: Cliente

- utiliza la clase `ContentResolver`
- indica los permisos que requiere en el manifiesto
- recuperación datos: método `query()`
  - ```
    Cursor query (Uri uri,
                  String[] projection,
                  String selection,
                  String[] selectionArgs,
                  String sortOrder);
    ```
  - devuelve un `Cursor` al resultado
- `ContentResolver` dispone además de métodos `insert()`, `update()`, `delete()`
- tipos MIME: `vnd.android.cursor.dir/<entidad>`, `vnd.android.cursor.item/<entidad>`

http://developer.android.com/reference/android/content/ContentResolver
    ```


[MiW - FEM] >
22
Atribución
Portions of this document are reproduced from work created and shared by the Android Open Source Project (https://source.android.com/) and used according to terms described in the Creative Commons 2.5 Attribution License (http://creativecommons.org/licenses/by/2.5/).
https://developer.android.com/license